NOVA School of Science and Technology



Systems Privacy

79262 79418

Professor
Kevin Gallagher

October 29, 2026

1. Lab 1: Implementing Contextual Integrity

1.1 Objective

The objective of this laboratory is to add Contextual Integrity to the DataSafe system using XACML access-control policies. The system decides whether access should be allowed based on the user's role, the type of data being accessed, and the current context.

The access rules are defined in `ci_policy.xml` and evaluated by the Policy Decision Point (PDP), keeping the access-control logic separate from the Java application.

1.2 `ci_policy.xml` file

We started by completing the `ci_policy.xml` file, which defines the access-control policies used by the system. Each policy contains a *Target*, which determines the requests to which the policy applies, and, when necessary, a *Condition*, which specifies the additional constraints that must be satisfied for the rule to return `Permit`.

The attributes used by the policies include the requester's role and identifier, the identifier and type of the requested resource, the doctor associated with a patient, and the context in which the access is performed. Figure 1.1 summarizes the policies implemented in the file.

Patient
PatientOwnData
Target: `role` = `Patient`
Condition: `subject.id` = `resource.id`

Employee
EmployeeOwnData
Target: `role` ∈ {Clerk, Accountant, Doctor, Nurse, ITAdmin}
Condition: `subject.id` = `resource.id`

Clerk
ClerkAppointmentData
Target: `role` = `Clerk`
Condition: `resource-type` = `AppointmentData` ∧ `context` ∈ {Scheduling, Check-in}

Accountant
AccountantBillingData
Target: `role` = `Accountant`
Condition: `resource-type` ∈ {PaymentData, EmployeeData} ∧ `context` = `BillingAndPayment`

Doctor / Nurse
DoctorOwnPatientsNonEmergency
Target: `role` ∈ {Doctor, Nurse}
Condition: `subject.id` = `resource.doctor-id` ∧ `resource-type` = `PatientData`

Doctor / Nurse
DoctorEmergency
Target: `role` ∈ {Doctor, Nurse}
Condition: `context` = `Emergency` ∧ `resource-type` = `PatientData`

IT Admin
ITAdminEmergency
Target: `role` = `ITAdmin` ∧ `context` = `Emergency`
Condition: not required

IT Admin
ITAdminHiring
Target: `role` = `ITAdmin`
Condition: `resource-type` = `EmployeeData` ∧ `context` = `Hiring`

Figure 1.1: Summary of the policies defined in `ci_policy.xml`, showing the Target and Condition of each policy.

1.3 `XACMLPDP.java` file

Then we moved on for the next job on our lab. The `XACMLPDP.java` file connects the Java application to the XACML policy. The `checkAccess()` method receives a requester, a resource,

and an access context, and converts them into the attributes required by `ci_policy.xml`, as shown in Figure 1.2.

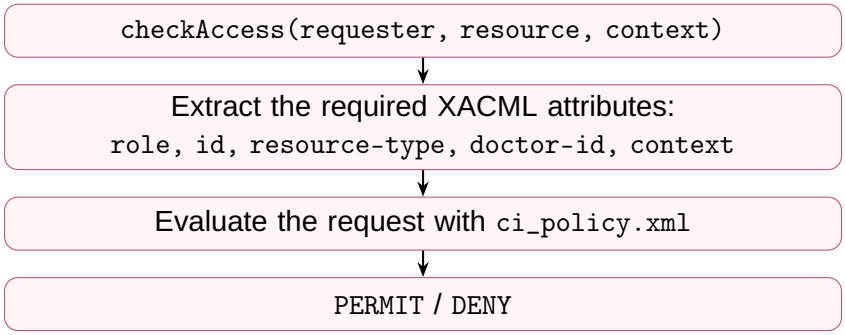


Figure 1.2: Access evaluation flow in `XACMLPDP.java`.

The extracted attributes depend on the type of object being processed (Figure 1.3). A requester is represented by its `id` and `role`. For resources, additional information may be necessary. A `Patient` includes its identifier, the `PatientData` resource type, and the identifier of its primary doctor. An `Appointment` is represented as `AppointmentData` and may also include the associated patient, doctor, and employee identifiers. A `Payment` is represented as `PaymentData` and uses the patient's identifier.

Requester

Patient → `id, role=Patient`
Employee → `id, role`

Resource

Employee → `id, EmployeeData`
Patient → `id, PatientData, doctor-id`
Appointment → `AppointmentData, id, doctor-id, employee-id`
Payment → `PaymentData, id`

Figure 1.3: Main attributes extracted from Java objects for the XACML request.

The access context is stored as an environment attribute, while the action is defined as `read`. The `addAttributes()` method converts these Java values into XACML attribute bags and adds them to the request.

Finally, the PDP evaluates the request against `ci_policy.xml`. The method returns `true` when the result is `PERMIT` and `false` otherwise.

1.4 Testing and Validation

The implemented access-control mechanism was tested with different users, resources, and contexts in order to verify the XACML policies.

Figure 1.4 shows an example where the patient *John W. Miller* accesses his own patient data. Since the subject and resource identifiers are both `1001`, the `PatientOwnData` policy applies and the PDP returns `PERMIT`.

```
SUBJECT = {role=Patient, id=1001}
RESOURCE = {doctor-id=2, resource-type=PatientData, id=1001}
ENVIRONMENT = {context=Appointment}
ACTION = {action-id=read}
DECISION = PERMIT
FULL RESULT = Result( decision=PERMIT, status=Optional.empty, pepActions=[], applicablePolicies= [Policy[PatientOwnData#v1.0], PolicySet[HospitalPolicySet#v1.0]] )
Access Granted!
```

Figure 1.4: Example of a permitted access request evaluated by the PDP.

Additional tests were performed for the remaining roles and contexts, confirming that valid requests are permitted and unauthorized requests are denied.

2. Lab 2: Implementing GDPR Rights

2.1 Objective

This lab extends DataSafe with two GDPR rights: *access* and *erasure*. Through the command-line interface, a user can request a JSON export of their personal data or request the anonymization of direct identifiers. Both operations build on the Hibernate database and the XACML access-control mechanism from Lab 1.

2.2 Database

Two Hibernate entities were introduced: `ConsentRecord`, stored in `consent_records`, and `GDPRRequest`, stored in `gdpr_requests`. Consent must be represented as an *append-only history*: granting or withdrawing consent produces a new record rather than replacing previous decisions. The current consent state is derived from the latest record for each purpose. GDPR requests store the request type, timestamps, status and subject identifier for accountability.

Both `Patient` and `Employee` now track `createdAt`, `updatedAt` and `retentionUntil`. They also maintain `consentRecords` and `gdprRequests` lists. For erasure, `Patient` additionally tracks the request flag and completion time, while `Payment` includes an `anonymized` flag.

The required test data covers active marketing consent, an erasure request and expired retention. Figure 2.1 shows the preparation of these scenarios.

```
INSERT INTO patients (name, birthdate, insurance, address, retention_until)
VALUES (
    'Test Patient - Marketing Consent',
    strftime('%s', '1990-01-01 00:00:00') * 1000,
    'Test insurance',
    'Test address',
    strftime('%s', 'now', '+1 year') * 1000
);

INSERT INTO consent_records
    (patient_subject_id, subject_type, purpose, granted, granted_at,
     withdrawn_at, policy_version, source)
VALUES (
    (SELECT id FROM patients WHERE name = 'Test Patient - Marketing Consent'),
    'PATIENT', 'marketing', 1, strftime('%s', 'now') * 1000,
    NULL, '1.0', 'test'
);

INSERT INTO patients (name, birthdate, insurance, address, retention_until)
VALUES (
    'Test Patient - Erasure Request',
    strftime('%s', '1990-01-01 00:00:00') * 1000,
    'Test insurance',
    'Test address',
    strftime('%s', 'now', '+1 year') * 1000
);

INSERT INTO gdpr_requests
    (patient_subject_id, subject_type, request_type, requested_at, status)
VALUES (
    (SELECT id FROM patients WHERE name = 'Test Patient - Erasure Request'),
    'PATIENT', 'ERASURE', strftime('%s', 'now') * 1000, 'PENDING'
);

INSERT INTO patients (name, birthdate, insurance, address, retention_until)
VALUES (
    'Test Patient - Expired Retention',
    strftime('%s', '1990-01-01 00:00:00') * 1000,
    'Test insurance',
    'Test address',
    strftime('%s', 'now', '-1 day') * 1000
);
```

Figure 2.1: Preparation of GDPR test data in the database.

2.3 XACML Policy

Five XACML 3.0 policies form the new `GDPRPolicySet`. At the root, `HospitalPolicySet` combines it with the Lab 1 policies. The `deny-overrides` algorithm gives priority to applicable GDPR denials.

For read requests, the six accepted lawful bases are consent, contract, legal-obligation, vital-interests, public-task and legitimate-interests. The Java PDP supplies consent status, erasure requests, retention expiry, data category and lawful-basis attributes to the XACML policies.

Policy	Access requirement
`ConsentPolicy`	Marketing requires active consent that has not been withdrawn.
`ErasurePolicy`	If erasure was requested, only the DPO is exempt from the access denial.
`RetentionPolicy`	If retention expired, only the Auditor and DPO are exempt from the access denial.
`SpecialCategoryPolicy`	Access to `health-data` requires research consent.
`LawfulBasisPolicy`	Read actions require a valid GDPR lawful basis.

Table 2.1: The five policies required for Lab 2.

2.4 DataSafe

Right to access. The `Export my data` option retrieves the authenticated subject's record and associated appointments, payments, consent records and GDPR requests. Before including each record, DataSafe calls `checkAccessGDPR(...)`. An authorized record is included in the JSON generated with Jackson; a denied record is represented by its identifier and `redacted = true`. Each access request is also logged in `gdpr_requests`. The export queries are scoped to the logged-in subject.

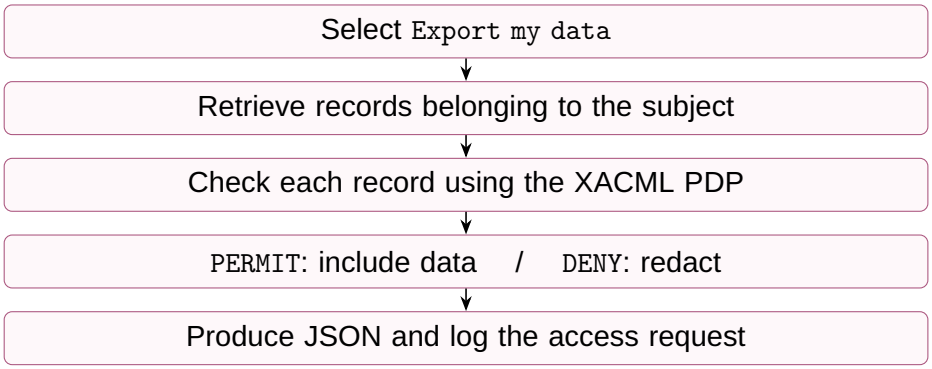


Figure 2.2: Right-to-access workflow in DataSafe.

Right to erasure. The `Request Erasure` option marks the patient's record as requested, inserts an `ERASURE` entry into `gdpr_requests`, and replaces direct identifiers (name, address and payment card numbers) with `ANON_<uuid>` tokens. Patient and payment rows are retained, payments are marked as anonymized, and the completion time is recorded. The operation uses a database transaction instead of physically deleting rows.

The patient's birth date remains unchanged. However, because the current implementation replaces the entire address, preserving a postal code embedded in that field is *not yet guaranteed*; this aspect still requires adjustment to meet the lab's quasi-identifier requirement.

2.5 Testing and Validation

The eight JUnit 5 policy tests passed (**8 tests, 0 failures, 0 errors**). They cover marketing consent, erasure restrictions, retention expiry and research consent for health data. The CLI screenshots in Figure 2.3 illustrate data export and an authorized record-access decision; they do not, by themselves, prove the erasure workflow.

3. Lab 3: Implementing K-Anonymity and T-Closeness

3.1 Objective

The goal of this lab is to anonymize medical measurements for research while reducing re-identification risks. We implemented a k-anonymity checker, the Mondrian partitioning algorithm and a variant that also enforces t-closeness. The GDPR policy for access to original

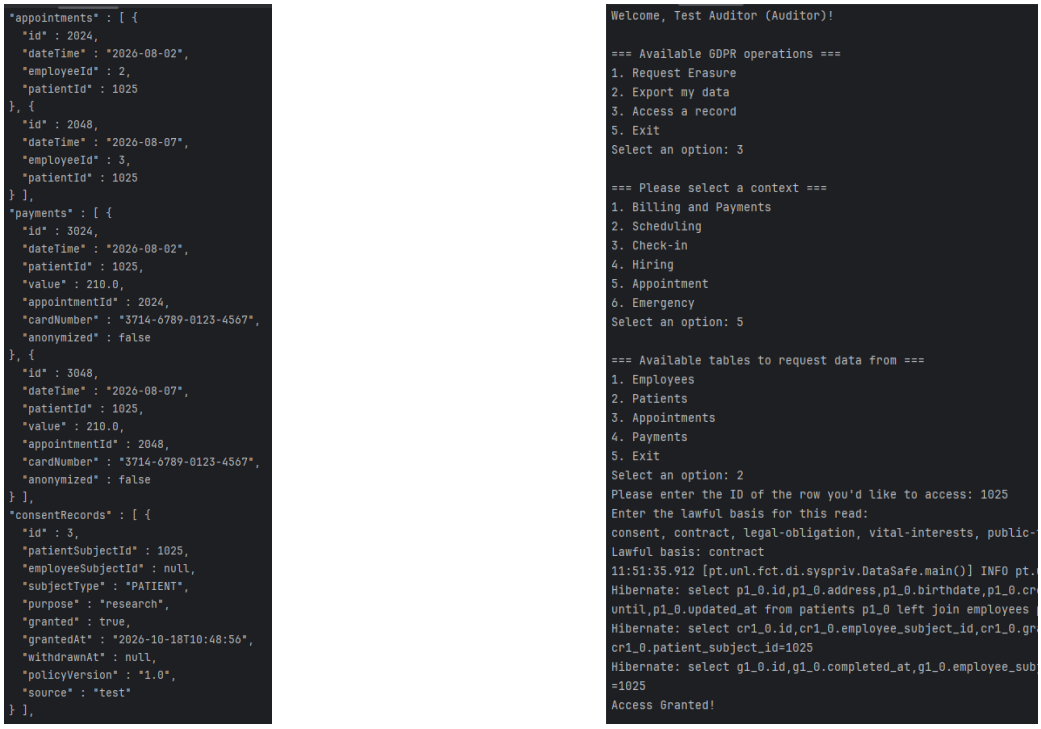


Figure 2.3: Examples of the DataSafe CLI: JSON export (left) and access authorization (right).

health records was also completed.

3.2 Database and Data Preparation

The provided CSV datasets were imported into the existing database, resulting in **150 patients**, **30 employees** and **300 health-data records**. The new Hibernate entity `HealthData` stores each patient's glucose measurement and its date.

The `AnonymousHealthData` class converts these records into a form without direct patient identifiers. Using the `Anonymizable` interface, it exposes three quasi-identifiers (birth year, postal code and diagnosis year) and the sensitive value (glucose level). No patient name or identifier is included in the anonymized representation.

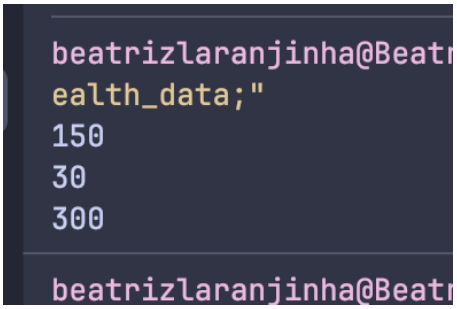


Figure 3.1: Database contents after importing the Lab 3 datasets.

3.3 K-Anonymity and Mondrian

In `KAnonymityTool.java`, `isKAnonymous()` groups records by identical quasi-identifiers and verifies that each group contains at least k records. The `runMondrain()` method recursively divides the dataset along numerical quasi-identifier dimensions, provided both resulting partitions contain at least k records. The final partitions are generalized into value ranges.

To enforce *t-closeness*, `runTCloseMondrain()` additionally checks each proposed partition against the overall glucose distribution using Earth Mover's Distance (EMD). A partition is split

only when both resulting groups respect the threshold t; otherwise, it is generalized without further splitting.

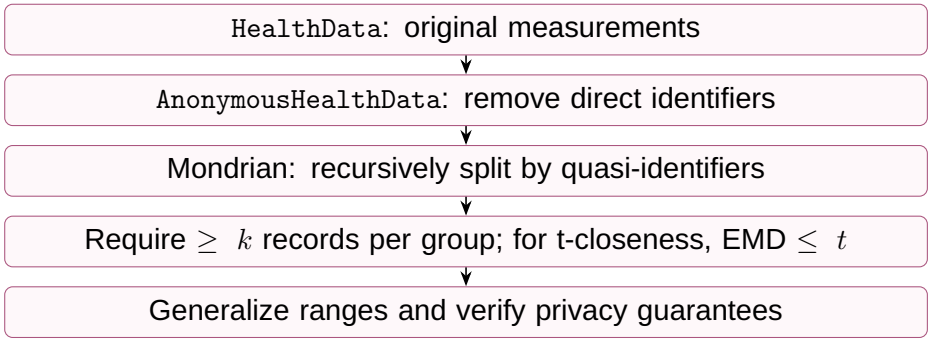


Figure 3.2: Anonymization workflow implemented in the Lab 3 tool.

3.4 GDPR Policy Integration

We updated `XACMLPDP.java` so that raw `HealthData` records have the `data-category` attribute set to `health-data`. The existing `SpecialCategoryPolicy` then requires active research consent (`consent-research`). A direct policy check confirmed that an otherwise authorized read is permitted with research consent and denied without it.

```
jshell> System.out.println(pdp.evaluateAccessRequest(subject, resource, action, env));
true

jshell> resource.put("consent-research", "false");
   ...>
   ...> System.out.println(pdp.evaluateAccessRequest(subject, resource, action, env));
$17 ==> "true"
false

jshell>
```

Figure 3.3: Research-consent policy verification: authorized and denied reads.

4. Lab 4: Implementing Differential Privacy

4.1 Objective

This lab implements Differential Privacy (DP) as a query layer to keep more utility. Two datasets are adjacent if one is obtained from the other by adding or removing one row, and a mechanism M is ε-differentially private if $\Pr[M(x) \in S] \leq e^{\varepsilon} \Pr[M(x') \in S]$ for every set of outputs S. We use the Laplace mechanism, which returns $F(x) + \mathrm{Lap}(s/\varepsilon)$ for a query with L_1 sensitivity s. Each answer spends ε from a privacy budget, and a query is rejected if the budget is insufficient.

4.2 DifferentialPrivacyTool.java file

This class is the query layer: researchers never see rows, only noisy aggregates whose total leakage is bounded by the budget (Figure 4.1). It supports four queries:

- **Count:** $s = 1$, since one row changes the count by at most one.
- **Sum:** values are clipped to $[0, 500]$ (configurable), so $s = 500$, the largest absolute value a row can contribute.
- **Average:** a noisy sum ($s = 500$) and a noisy count ($s = 1$), each with $\varepsilon/2$, for a total of ε by sequential composition. The denominator is bounded below by 1.

- **Histogram:** public, fixed age $\times$ glucose bins, *including empty ones*, each with Laplace noise of scale $1/\varepsilon$. A row falls in exactly one bin, so the L_1 sensitivity is 1 and the whole histogram costs ε (not one ε per bin).

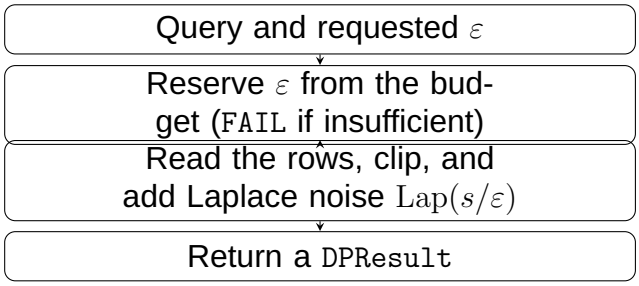


Figure 4.1: Processing of a differentially private query.

The cost is subtracted *before* the data is read, so a failed query can never leak without being accounted for, and the budget never becomes negative. The noise is generated from a uniform value with the inverse CDF. The source of randomness is injectable (`SecureRandom` in production, fixed values in the tests) and the value is kept away from the endpoints so the noise is always finite. The method `split` divides the sorted record IDs into two disjoint subsets, charges the allowance once to the parent, gives each subset the full allowance, and blocks further queries on the parent.

4.3 DPResult.java and config.props files

`DPResult` carries the noisy value (or histogram), the query sensitivity, the clipping bounds, the remaining budget and a `SUCCESS`/`FAIL` code. All parameters (initial budget, default ε and the clipping bounds for glucose and age) are read from `config.props`, so they can change without recompiling.

4.4 RegressionUtilityExperiment.java file

The experiment fits glucose against age with the provided `LinearRegression` on the 300 records in four settings: unprotected, k-anonymity ($k = 5$), k-anonymity with t-closeness ($t = 0.2$), and DP. For DP we release one noisy histogram with 10-year $\times$ 50-unit bins ($13 \times 11 = 143$ bins) for $\varepsilon \in \{0.1, 0.5, 1.0\}$, and expand each bin into points at its midpoint, with negative counts rounded to zero (post-processing, no extra cost). It also runs a composition experiment, using its own in-memory budget (2.0) so it does not consume the budget of the menu, and prints the budget actually deducted.

4.5 DataSafe.java file

Two menu options were added: DP queries and the experiment above. Both are restricted to the DPO role, because the budget is shared and any user could otherwise exhaust it.

4.6 Testing and Validation

The JUnit 5 tests check that: with fixed uniform values the noise matches the Laplace formula, and over 20,000 samples its mean and variance are close to 0 and $2s^2/\varepsilon^2$; queries fail without changing the budget when it is insufficient, the budget is exhausted without going negative, and invalid ε values are rejected; values are clipped and the sum sensitivity is correct; `split` gives the full allowance to each subset and rejects invalid cuts, a second split and queries on the parent; and, with zero noise, the DP results match the SQL results on the 300 records.

4.7 Results

Table 4.1 shows the regression in each setting for one execution. The relationship is weak even on the original data: age explains only about 10% of the variance of glucose.

Setting	n	Slope	Intercept	R^2	MSE
Unprotected	300	1.045	86.19	0.096	1759.6
k-anonymity ($k = 5$)	300	1.075	84.10	0.096	1758.9
k-anon. + t-closeness	300	0.997	87.58	0.083	1785.0
DP, $\varepsilon = 0.1$	735	1.153	160.00	0.061	19950.6
DP, $\varepsilon = 0.5$	431	1.252	106.10	0.084	11515.4
DP, $\varepsilon = 1.0$	346	1.515	85.56	0.125	8039.0

Table 4.1: Linear regression of glucose on age under each protection (one run).

In the composition experiment, a count query (glucose < 120) with $\varepsilon = 0.1$ was run once on each of two disjoint halves of the sorted IDs (combined estimate 143.3) and twice on the full data (estimates 122.3 and 102.5). The experiment deducted 1.9 of 2.0 from the budget: 1.6 for the histograms, 0.1 for the parallel part and 0.2 for the sequential part.

4.8 Discussion

k-anonymity and t-closeness. k-anonymity barely changes the result (slope difference 0.030): generalising birth and diagnosis years keeps the age well and leaves the glucose untouched, but it does not bound how much the output changes when one row is added, and it can be vulnerable to background knowledge. t-closeness merges classes to keep the glucose distribution close to the global one, which costs more utility (slope difference 0.048, MSE $+25.4$) in exchange for protecting the sensitive attribute.

Differential privacy. It distorts the most. With 300 records in 143 bins there are about two records per bin, and the noise scales (10, 2 and 1) are comparable to the signal. Rounding negative noisy counts to zero inflates the sample ($n = 735, 431, 346$). The intercept error shrinks as ε grows (73.8, 19.9, 0.62), but the slope error does not (0.107, 0.207, 0.470), which is within the randomness of a single run. We chose $\varepsilon = 0.1$, 0.5 and 1.0 for strong, moderate and weak privacy; $\varepsilon = 0.1$ was unusable and none was clearly best in one run.

Composition. For a fixed partition independent of the data, parallel composition costs $\max(0.1, 0.1) = 0.1$, while the two sequential queries cost $0.1 + 0.1 = 0.2$. Repeating a query is also wasteful: for $s = 1$ the variance is $2/\varepsilon^2$, so averaging two answers with $\varepsilon = 0.1$ gives variance 100, while one answer with $\varepsilon = 0.2$ gives 50.

Conclusion. For these 300 records, k-anonymity gives the best utility and t-closeness adds protection of the sensitive attribute. DP gives a different, formal guarantee, but here its utility is dominated by noise and coarse binning, and it would suit larger datasets better.